COURSE PRACTICE / 2026.11

微处理器

填空与编程练习册

104 道分章填空题 · 23 道汇编程序填空题 · 共 303 空

依据：2026 版课本、课程课件、往届带答案原卷、课堂记录与三次实验。重点参照 2026 年 10 月 19 日两份 Lec12 复习记录。

- 只编入普通填空和汇编程序填空，不含简答题。第二章作为读硬件的基础，编程不要求编写Verilog。
- 主线依据2026-10-19两份重点记录；复习提点不等于确定考点。本套为学习练习，不能预测试卷。
- 教材页码为正文印刷页，PDF页序=正文页+5。课件页码直接按PDF页序。每题保留来源。
- 题目用ARMASM课堂风格，面向Cortex-M0；程序变量、地址、电平和协议均以各题题面为准。
- 自动核对只识别参考答案及已列同义写法，不证明程序语义；等价写法可对照解析后自评。
- 已校订课件和现有试卷整理中的明显问题：软核“实现工艺”、F(50)溢出、#R0笔误、C调用寄存器保护、栈对齐、字符串容量与M0访存写法。
- 专项组卷只含15个普通填空+5/10/10个程序空，共80分，不包含原考试20分简答，不能作为完整模拟卷。

练习顺序

先按教材章节查漏，再练 P01-P05 纯汇编、P06-P11 混合编程、P12-P18 软硬件综合，以及新增的 P19-P23 C 调用汇编迁移题。C调用示例已经给出，只填汇编。答案编号与网页一致。

打印建议：练习册与答案册分开装订。程序空按题面指令格式填写；答案册提供逐空参考、边界案例和语义解析。

目录与专项组卷

第1章 片上系统设计方法概述 10题 / p3

第2章 硬件描述语言与数字系统基础 10题 / p5

第3章 微处理器原理与 ARM 内核 22题 / p7

第4章 Thumb 指令与 ARM 汇编程序设计 22题 / p10

第5章 Cortex-M0 片上系统与总线 20题 / p13

第6章 IP 集成与软硬件协同 20题 / p16

程序填空

纯汇编 P01-P05 / p19

C 与汇编混合编程 P06-P11 / p24

软硬件综合与接口驱动 P12-P18 / p30

C调用汇编迁移 P19-P23 / p37

4 套专项练习 · 每套 80 分

每套由 15 个普通填空和 3 道程序题组成，程序空数为 5/10/10。每空 2 分。仅为本题库的组合，不包含简答题，也不代表完整考试。

专项 A · 基础与 GPIO

填空：F1-04、F1-09、F3-03、F3-06、F3-08、F3-13、F3-15、F3-16、F4-15、F4-21、F5-08、F5-11、F5-14、F6-02、F6-08

编程：P01、P08、P13

专项 B · 调用与显示

填空：F1-03、F2-03、F3-07、F3-09、F3-14、F3-17、F3-20、F4-13、F4-16、F4-20、F5-07、F5-15、F5-18、F6-10、F6-13

编程：P03、P09、P15

专项 C · 综合与加速器

填空：F1-06、F2-10、F3-05、F3-12、F3-18、F4-08、F4-11、F4-22、F5-09、F5-10、F5-17、F5-20、F6-15、F6-16、F6-18

编程：P02、P10、P16

专项 D · C调用汇编迁移

填空：F3-08、F3-09、F3-10、F3-17、F3-18、F4-03、F4-04、F4-09、F4-10、F4-11、F4-12、F4-15、F4-20、F4-21、F4-22

编程：P19、P22、P20

资料回查

普通填空题后的 p 为教材正文页，PDF 页序需加 5。课件参考写明 PDF 页序。实验路径和课堂时间戳可在网页中展开查看。

CHAPTER 01 / 分章填空

片上系统设计方法概述

F1-01 教材巩固

SoC 的中文名称是（1）_____。

来源：教材第1章 · 正文 p7-8

F1-02 教材巩固

购买处理器 IP 得到的是可集成的（1）_____，还须与存储器及外设等组合才能形成实际 SoC。

来源：教材第1章 · 正文 p12

F1-03 教材巩固

IP 核按交付形式分为软核、（1）_____和硬核。

来源：教材第1章 · 正文 p12

F1-04 复习课件例题

由于软核不依赖于最终的实现（1）_____，因此具有较大灵活性。

来源：教材第1章 · 正文 p12；期末复习总结 · 填空例题与答案

F1-05 教材巩固

软核通常以（1）_____源文件的形式交付。

来源：教材第1章 · 正文 p12

F1-06 教材巩固

将功能分配给软件或硬件实现，是软硬件协同设计中的（1）_____划分。

来源：教材第1章 · 正文 p9-10

F1-07 教材巩固

EDA 的中文含义是（1）_____。

来源：教材第1章 · 正文 p10-11

F1-08 教材巩固

从系统总体需求逐步分解到各模块的设计方法称为（1）_____设计。

来源：教材第1章 · 正文 p10-11

F1-09 教材巩固

多个处理器各有 Cache 而共享数据时，需要解决 Cache 的（1）_____问题。

来源：教材第1章 · 正文 p13-15

F1-10 教材巩固

多处理器共享总线时，同时访存会产生总线（1）_____，限制扩展性。

来源：教材第1章 · 正文 p13-15

CHAPTER 02 / 分章填空

硬件描述语言与数字系统基础

F2-01 教材巩固

Verilog 模块用 module 开始，用（1）_____结束。

来源：教材第2章 · 正文 p26-27

F2-02 教材巩固

组合逻辑中的连续赋值常使用关键字（1）_____。

来源：教材第2章 · 正文 p29-30

F2-03 教材巩固

在时序 always 块中，为表达寄存器同时更新，通常使用（1）_____赋值。

来源：教材第2章 · 正文 p30-31

F2-04 教材巩固

仅在 posedge clk 时更新的存储单元，其状态通常在时钟（1）_____改变。

来源：教材第2章 · 正文 p46-47

F2-05 教材巩固

always @(posedge clk or negedge rst_n) 中 rst_n 是低有效的（1）_____复位。

来源：教材第2章 · 正文 p46-47

F2-06 教材巩固

Moore 状态机的输出仅与（1）_____有关。

来源：教材第2章 · 正文 p85-86

F2-07 教材巩固

Mealy 状态机的输出与当前状态以及当前（1）_____有关。

来源：教材第2章 · 正文 p85-86

F2-08 教材巩固

数码管显示中，选择哪个数字图形的是（1）_____信号；选择哪一位亮的是（2）_____信号。

来源：教材第2章 · 正文 p65-70

F2-09 教材巩固

PWM 调光主要通过改变高电平持续时间与周期之比，即（1）_____，来改变平均亮度。

来源：教材第2章 · 正文 p87-91

F2-10 教材巩固

加速器硬件常划分为执行运算的（1）_____与决定各步动作的控制逻辑。

来源：教材第2章 · 正文 p109-116

CHAPTER 03 / 分章填空

微处理器原理与 ARM 内核

F3-01 教材巩固

冯·诺依曼计算机的五大部件为运算器、控制器、存储器、（1）_____和（2）_____。

来源：教材第3章 · 正文 p122-125

F3-02 教材巩固

计算机内部三总线是地址总线、数据总线和（1）_____总线。

来源：教材第3章 · 正文 p124-125

F3-03 教材巩固

指令的基本执行过程是取指、（1）_____和执行。

来源：教材第3章 · 正文 p128-129

F3-04 教材巩固

对程序员可见的机器属性，如指令系统，属于计算机的（1）_____。

来源：教材第3章 · 正文 p121-123

F3-05 教材巩固

指令系统是软件与（1）_____之间的约定。

来源：教材第3章 · 正文 p130-132

F3-06 教材巩固

机器指令中，指明完成何种操作的字段是（1）_____。

来源：教材第3章 · 正文 p130-132

F3-07 真题改编

一条指令的（1）_____和相关寻址方式会影响处理器的组织结构。

来源：教材第3章 · 正文 p130-132；往届 A 卷 · 填空题

F3-08 教材巩固

操作数直接包含在指令编码中的寻址方式是（1）_____寻址。

来源：教材第3章 · 正文 p134-135

F3-09 教材巩固

操作数在指定通用寄存器内的方式称为（1）_____寻址。

来源：教材第3章 · 正文 p135

F3-10 教材巩固

寄存器内存放操作数的存储地址，按该地址访存，称为寄存器（1）_____寻址。

来源：教材第3章 · 正文 p135-136

F3-11 教材巩固

以 PC 为基准加偏移形成转移目标的方式称为（1）_____寻址。

来源：教材第3章 · 正文 p138

F3-12 教材巩固

RISC 的访存通常采用（1）_____体系，仅加载与存储指令访问存储器。

来源：教材第3章 · 正文 p139-140

F3-13 教材巩固

Cortex-M0 中 R13、R14、R15 分别称为 SP、（1）_____、PC。

来源：教材第3章 · 正文 p148-149

F3-14 教材巩固

执行 BL 或 BLX 调用时，函数返回地址被写入（1）_____。

来源：教材第3章 · 正文 p149

F3-15 真题改编

Cortex-M0 中 EPSR 的 T 位表示（1）_____状态。

来源：教材第3章 · 正文 p150；往届 A 卷 · 填空题

F3-16 真题改编

EPSR 的 T 位被置零会在后续执行时引发（1）_____异常。

来源：教材第3章 · 正文 p150；往届 A 卷 · 填空题

F3-17 教材巩固

APSR 的 N、Z、C、V 中，表示结果为零的标志是（1）_____。

来源：教材第3章 · 正文 p150

F3-18 教材巩固

APSR 中表示有符号运算溢出的标志为（1）_____。

来源：教材第3章 · 正文 p150

F3-19 教材巩固

按本课程 Cortex-M0 的字节序，0x12345678 存到地址 A 后，最低地址字节为 0x（1）_____。

来源：教材第3章 · 正文 p153

F3-20 教材巩固

Cortex-M0 栈采用（1）_____模型，PUSH 后栈向低地址增长。

来源：教材第3章 · 正文 p137-138

F3-21 教材巩固

Cortex-M0 的硬件异常处理使用（1）_____栈指针。

来源：教材第3章 · 正文 p149

F3-22 复习课件例题

程序倾向于再次访问最近访问的数据或指令，复习课件例题将其称为（1）_____局部性。

来源：教材第3章 · 正文 p125-126；期末复习总结 · 填空例题与答案

CHAPTER 04 / 分章填空

Thumb 指令与 ARM 汇编程序设计

F4-01 教材巩固

把存储器中的一个字加载到寄存器，常使用（1）_____指令。

来源：教材第4章 · 正文 p175-177

F4-02 教材巩固

将寄存器的一个字写入存储器，常使用（1）_____指令。

来源：教材第4章 · 正文 p175-177

F4-03 教材巩固

读取一个无符号字节并零扩展，常使用（1）_____指令。

来源：教材第4章 · 正文 p175-177

F4-04 教材巩固

LDR R0,[R1,#4] 的有效地址为 R1 的值加（1）_____字节。

来源：教材第4章 · 正文 p175-177

F4-05 教材巩固

LSRS 右移时，高位补（1）_____。

来源：教材第4章 · 正文 p168-170

F4-06 教材巩固

ASRS 右移时，高位补原数据的（1）_____位。

来源：教材第4章 · 正文 p168-170

F4-07 教材巩固

实现按位与运算并更新标志的指令是（1）_____。

来源：教材第4章 · 正文 p166-168

F4-08 教材巩固

只做按位与测试、更新条件标志而不保存结果，可使用（1）_____。

来源：教材第4章 · 正文 p171-172

F4-09 教材巩固

CMP R0,#0 后，R0 为零时可使用（1）_____跳转。

来源：教材第4章 · 正文 p165-166

F4-10 教材巩固

CMP R0,#0 后，R0 非零时可使用（1）_____跳转。

来源：教材第4章 · 正文 p165-166

F4-11 教材巩固

比较两个无符号数，R0 小于 R1 时可用 BCC，也称（1）_____。

来源：教材第4章 · 正文 p172-174

F4-12 教材巩固

从不再调用其他函数的叶子函数返回，可执行 BX（1）_____。

来源：教材第4章 · 正文 p172-174

F4-13 教材巩固

在函数中执行另一次 BL 之前，通常要保存当前（1）_____，否则会丢失返回路径。

来源：教材第4章 · 正文 p173-174

F4-14 教材巩固

汇编源程序中定义代码段或数据段，ARMASM 使用（1）_____伪指令。

来源：教材第4章 · 正文 p193-194

F4-15 教材巩固

向其他源文件公开当前文件定义的函数符号，使用（1）_____伪指令。

来源：教材第4章 · 正文 p194-195

F4-16 教材巩固

汇编代码调用外部 C 函数前，可用（1）_____声明外部符号。

来源：教材第4章 · 正文 p194-195

F4-17 教材巩固

ARMASM 定义一个 32 位常数数据项使用（1）_____。

来源：教材第4章 · 正文 p189

F4-18 教材巩固

ARMASM 为栈等区域保留字节空间，可使用（1）_____。

来源：教材第4章 · 正文 p189

F4-19 教材巩固

LDR R0,=0x40000000 中带等号的形式属于加载常数的（1）_____指令。

来源：教材第4章 · 正文 p181-182

F4-20 教材巩固

按课程常用调用规则，前四个整型参数依次放在 R0、R1、R2、（1）_____。

来源：教材第4章 · 正文 p207-209

F4-21 教材巩固

普通 32 位整型函数返回值通常放在（1）_____寄存器。

来源：教材第4章 · 正文 p207-209

F4-22 教材巩固

函数借用 R4-R7 等被调用者保存寄存器后，返回前必须将它们（1）_____。

来源：教材第4章 · 正文 p207-209

CHAPTER 05 / 分章填空

Cortex-M0 片上系统与总线

F5-01 真题改编

在典型主从式总线系统中，处理器通常作为（1）_____方主动发起传输。

来源：教材第5章 · 正文 p217-220；往届 A 卷 · 填空题

F5-02 教材巩固

地址总线在本课程单主机系统中，由（1）_____发送地址到从机。

来源：教材第5章 · 正文 p217-220

F5-03 教材巩固

同步总线使用共同的（1）_____信号协调传输。

来源：教材第5章 · 正文 p221-224

F5-04 教材巩固

异步总线主要通过请求与应答等（1）_____信号协调传输。

来源：教材第5章 · 正文 p223-224

F5-05 教材巩固

半同步总线可通过插入（1）_____状态适应慢设备。

来源：教材第5章 · 正文 p224

F5-06 复习课件例题

AHB-Lite 传输分为（1）_____相位与（2）_____相位。

来源：教材第5章 · 正文 p226-227；期末复习总结 · 填空例题与答案

F5-07 教材巩固

AHB-Lite 的 HWRITE=1 表示（1）_____操作。

来源：教材第5章 · 正文 p225-226

F5-08 真题改编

AHB-Lite 的 HREADY=0 表示传输进入（1）_____状态。

来源：教材第5章 · 正文 p226-227；往届 A 卷 · 填空题

F5-09 教材巩固

HTRANS 的 NONSEQ 编码为二进制（1）_____。

来源：教材第5章 · 正文 p225-226

F5-10 教材巩固

在 AHB-Lite 中，32 位字传输对应 HSIZE 的十进制值（1）_____。

来源：教材第5章 · 正文 p225-226

F5-11 真题改编

地址空间是地址（1）_____的集合。

来源：教材第5章 · 正文 p228-229；往届 A 卷 · 填空题

F5-12 教材巩固

32 位字节地址能够表示的地址空间大小为（1）_____GiB。

来源：教材第5章 · 正文 p229

F5-13 教材巩固

存储器与 I/O 使用同一套地址编码，称为（1）_____编址。

来源：教材第5章 · 正文 p228-229

F5-14 真题改编

统一编址使 I/O 访问可以使用较丰富的（1）_____。

来源：教材第5章 · 正文 p229；往届 A 卷 · 填空题

F5-15 教材巩固

在本课程未重定位的向量表中，地址 0x00000000 保存初始（1）_____值。

来源：教材第5章 · 正文 p230,239-240

F5-16 复习课件例题

向量表基址为 0 时，HardFault 异常向量项位于 0x（1）_____。

来源：教材第5章 · 正文 p230；期末复习总结 · 填空例题与答案

F5-17 教材巩固

地址译码电路将地址编码转换成相应模块的（1）_____信号。

来源：教材第5章 · 正文 p231-234

F5-18 实验二

若 HADDR[31:4]==28'h4000001，则该外设的起始字节地址是 0x（1）_____。

来源：教材第5章 · 正文 p231-234；实验二要求；实验二寄存器与电平定义

F5-19 实验二

上述 16 字节窗口内，偏移 0x04 的 32 位寄存器地址为 0x（1）_____。

来源：教材第5章 · 正文 p231-234；实验二要求；实验二寄存器与电平定义

F5-20 实验三

AHB 写操作中，应先锁存地址相位信息，再在对应的（1）_____相位用 HWDATA 写目标寄存器。

来源：教材第5章 · 正文 p226-227；实验三要求；实验三加速器寄存器接口

CHAPTER 06 / 分章填空

IP 集成与软硬件协同

F6-01 教材巩固

软件直接访问的接口寄存器，按作用通常分为数据端口、状态端口和（1）_____端口。

来源：教材第6章 · 正文 p258-259

F6-02 复习课件例题

程序查询方式在传输前由 CPU 读取接口的（1）_____寄存器。

来源：教材第6章 · 正文 p262；期末复习总结 · 填空例题与答案

F6-03 真题改编

查询传输的接口硬件至少需要（1）_____端口和（2）_____端口。

来源：教材第6章 · 正文 p262；往届 A 卷 · 填空题

F6-04 教材巩固

外设始终可接受访问、不需先检查状态的方式叫（1）_____传送。

来源：教材第6章 · 正文 p259,261-262

F6-05 教材巩固

外设通过请求 CPU 暂停当前程序并执行服务程序来传输，属于程序（1）_____方式。

来源：教材第6章 · 正文 p263-266

F6-06 教材巩固

中断前程序正在使用的状态标志及相关寄存器内容称为（1）_____。

来源：教材第6章 · 正文 p264

F6-07 教材巩固

中断返回时主程序继续执行的位置称为中断（1）_____。

来源：教材第6章 · 正文 p264

F6-08 真题改编

DMA 传完一批数据后常用（1）_____方式通知 CPU。

来源：教材第6章 · 正文 p267-269；往届 A 卷 · 填空题

F6-09 实验二

数码管动态显示通过依次选择不同（1）_____，利用视觉暂留显示多位数字。

来源：教材第6章 · 正文 p270-273；实验二要求；实验二寄存器与电平定义

F6-10 实验二

本实验二段选为高有效，位选为（1）_____有效。

来源：教材第6章 · 正文 p270-273；实验二要求；实验二寄存器与电平定义

F6-11 实验二

实验二关闭四位数码管时，位选寄存器低四位应写 0x（1）_____。

来源：教材第6章 · 正文 p270-273；实验二要求；实验二寄存器与电平定义

F6-12 实验二

动态扫描改变段码前先关闭位选，主要用于减少显示（1）_____。

来源：教材第6章 · 正文 p270-273；实验二要求；实验二寄存器与电平定义

F6-13 实验二

读取 8 位开关后，若输入为二进制 10000000，十进制显示应为（1）_____。

来源：教材第6章 · 正文 p270-273；实验二要求；实验二寄存器与电平定义

F6-14 实验三

加速器驱动通过写（1）_____寄存器发出启动、清除等命令。

来源：教材第6章 · 正文 p289-295；实验三要求；实验三加速器寄存器接口

F6-15 实验三

实验三 STATUS 中 BUSY、DONE、ERROR 依次位于 bit0、bit1、bit（1）_____。

来源：教材第6章 · 正文 p289-295；实验三要求；实验三加速器寄存器接口

F6-16 实验三

实验三 CTRL 的 ABORT 命令写入数值为（1）_____。

来源：教材第6章 · 正文 p289-295；实验三要求；实验三加速器寄存器接口

F6-17 实验三

实验三 SIN_RESULT 的字节偏移为 0x（1）_____。

来源：教材第6章 · 正文 p289-295；实验三要求；实验三加速器寄存器接口

F6-18 实验三

实验三驱动轮询必须设置次数上限，以避免外设故障导致程序（1）_____。

来源：教材第6章 · 正文 p289-295；实验三要求；实验三加速器寄存器接口

F6-19 实验三

实验三的 sin/cos 结果隐含三位小数，显示 0866 表示数值（1）_____。

来源：教材第6章 · 正文 p289-295；实验三要求；实验三加速器寄存器接口

F6-20 实验三

实验三定点后处理使用（1）_____位中间值来完成乘 1000 等计算与舍入。

来源：教材第6章 · 正文 p289-295；实验三要求；实验三加速器寄存器接口

P01 / 纯汇编 / 5 空 / 真题改编

统计二进制 1 的个数

uint32_t count_ones(uint32_t n)：R0 传入任意 32 位无符号数，R0 返回其二进制 1 的个数。将往届固定 n=14 的代码改为可调用函数。

C调用示例（已给）

```
extern uint32_t count_ones(uint32_t n);
uint32_t bits = count_ones(0x80007001u); /* bits=5 */
```

```
AREA Count,CODE,READONLY
        THUMB
        EXPORT count_ones
count_ones PROC
        MOVS R1,#0
loop    CMP R0,#0
        BEQ (1)_______
        MOVS R2,R0
        MOVS R3,#1
        ANDS R2,(2)_______,R3
        BEQ skip
        ADDS (3)_______,R1,#1
skip    LSRS R0,R0,(4)_______
        B (5)_______
done    MOVS R0,R1
        BX LR
        ENDP
        END
```

来源：往届 A 卷 · 编程 1；教材 §4.1 · 位运算与移位；用户补充参考图5 · 另一种位统计约定；用户补充参考图6 · 掩码扫描，可与P21对照

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____

寄存器规划 / 调试记录：

P02 / 纯汇编 / 5 空 / 课件改编

寻找 16 位最高有效位

输入 R0 为 1-65535，返回最高位 1 的位序到 R0（最低位编号 0）。用 R1=15 从高位向低位扫描。

```
  AREA MSB,CODE,READONLY
         THUMB
         EXPORT highest_bit
  highest_bit PROC
         MOVS R1,(1)_______
         LDR R2,(2)_______
  scan   MOVS R3,R0
         ANDS R3,R3,R2
         BNE (3)_______
         SUBS R1,R1,#1
         LSRS R2,R2,(4)_______
         B (5)_______
  found  MOVS R0,R1
         BX LR
         ENDP
         END
```

来源：期末复习总结 · 最高位例题

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____

寄存器规划 / 调试记录：

P03 / 纯汇编 / 5 空 / 实验一改编

拆分半字并按指定地址存储

实验一的安全化变式：R0 为 32 位输入，高 16 位为 A，低 16 位为 B；R1 为有效且 4 字节对齐的目标 RAM 指针。计算 A+B×A，写入 *R1 并从 R0 返回（保留低 32 位）。

```
  AREA Pack,CODE,READONLY
         THUMB
         EXPORT calc_store
calc_store PROC
         LSRS R2,R0,(1)______
         UXTH R3,(2)_______
         MULS R3,(3)_______,R3
         ADDS R0,R2,R3
         STR R0,(4)_______
         BX (5)_______
         ENDP
         END
```

来源：实验一 · 任务1；教材 §4.1 · 算术和访存

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____

寄存器规划 / 调试记录：

P04 / 纯汇编 / 5 空 / 教材变式

累加数组中的无符号字

uint32_t sum_words(const uint32_t *p,uint32_t n)。R0=p，R1=n；n=0 返回 0，否则累计 n 个字，按模 2^32 返回。指针有效且字对齐。

```
  AREA Sum,CODE,READONLY
         THUMB
         EXPORT sum_words
  sum_words PROC
         MOVS R2,#0
         CMP R1,#0
         BEQ finish
  loop   LDR R3,(1)_______
         ADDS R2,R2,(2)_______
         ADDS R0,R0,(3)_______
         SUBS R1,R1,#1
         (4)_______ loop
  finish  MOVS R0,(5)_______
         BX LR
         ENDP
         END
```

来源：教材 §4.2.3 · 循环结构；复习记录 · 37:45-44:22 程序填空要求

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____

寄存器规划 / 调试记录：

P05 / 纯汇编 / 5 空 / 教材变式

减法式最大公约数

uint32_t gcd_sub(uint32_t a,uint32_t b)。R0=a、R1=b，均为 1-65535；用反复相减求 GCD，R0 返回结果。比较均按无符号处理。

```
  AREA Gcd,CODE,READONLY
         THUMB
         EXPORT gcd_sub
gcd_sub PROC
loop     CMP R0,R1
         BEQ (1)_______
         (2)_______ less
         SUBS R0,R0,(3)_______
         B loop
less     SUBS R1,R1,(4)_______
         B (5)_______
done     BX LR
         ENDP
         END
```

来源：教材 §2.5.1 · GCD 算法；教材 §6.3.2 · GCD 加速器

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____

寄存器规划 / 调试记录：

P06 / 混合编程 / 10 空 / 复习课件改编

C 调汇编：斐波那契

C 原型 uint32_t fibonacci(uint32_t a,uint32_t b,uint32_t n)。a=b=1，2≤n≤46；R0/R1/R2 传参，返回第 n 项。补全 ARMASM 结构和递推循环。

C调用示例（已给）

```
extern uint32_t fibonacci(uint32_t a,uint32_t b,uint32_t n);
uint32_t f = fibonacci(1u,1u,10u); /* f=55 */
```

```
(1)_______ Fib,CODE,READONLY
        THUMB
        (2)_______ fibonacci
fibonacci PROC
        SUBS R2,R2,(3)_______
        BEQ done
loop    ADDS R3,R0,(4)_______
        MOVS R0,(5)_______
        MOVS R1,(6)_______
        SUBS R2,R2,(7)_______
        (8)_______ loop
done    MOVS R0,(9)_______
        BX (10)_______
        ENDP
        END
```

来源：期末复习总结 · Fibonacci 例题；重点复习记录 · 01:14:50-01:21:46

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：

P07 / 混合编程 / 10 空 / 课件校订改编

汇编调用 C：两日期间隔

C 函数 int date_index(int year,int month,int day) 已实现，year=年份后两位；返回可相减的日期序号。汇编函数 gap_days(void) 计算 2023-06-15 减去 2000-01-01 的天数。C 函数按调用约定可改写 R0-R3。

```
(1)______ DateGap,CODE,READONLY
        THUMB
        (2)______ date_index
        EXPORT gap_days
gap_days PROC
        PUSH {R4,(3)______}
        MOVS R0,(4)______
        MOVS R1,(5)______
        MOVS R2,(6)______
        (7)______ date_index
        MOVS R4,(8)______
        MOVS R0,#0
        MOVS R1,#1
        MOVS R2,#1
        BL date_index
        SUBS R0,(9)______,R0
        POP {R4,(10)______}
        ENDP
        END
```

来源：期末复习总结 · 日期混合编程例题；教材 §4.2.4 · 混合编程

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：

P08 / 混合编程 / 10 空 / 真题校订改编

递归阶乘与启动结构

补全原卷风格的启动框架与 uint32_t factorial(uint32_t n)。输入 0≤n≤12；复位后计算 5!。Stack_Size=0x400，栈段 ALIGN=3。

```
PRESERVE8
        THUMB
Stack_Size EQU 0x400
        AREA STACK,NOINIT,READWRITE,ALIGN=3
Stack_Mem (1)_______ Stack_Size
__initial_sp
        AREA RESET,DATA,READONLY
        EXPORT __Vectors
__Vectors DCD (2)_______
        DCD Reset_Handler
        AREA Fact,CODE,READONLY
        EXPORT factorial
Reset_Handler PROC
        MOVS R0,#5
        (3)_______ factorial
stop    B stop
        ENDP
factorial PROC
        PUSH {R4,(4)_______}
        MOVS R4,(5)_______
        (6)_______ R4,#1
        BLS base
        SUBS R0,(7)_______,#1
        BL (8)_______
        (9)_______ R0,R4,R0
        B ret
base    MOVS R0,#1
ret     POP {R4,(10)_______}
        ENDP
        END
```

来源：往届 A 卷 · 编程2；教材 §5.3.2 · 向量表与启动

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：

P09 / 混合编程 / 10 空 / 实验一改编

实验一：汇编调用累加 C 函数并求商

已给 C 函数 uint32_t add(uint32_t a)，返回 1+…+a。汇编函数 mean_sum(uint32_t a) 在 1≤a≤99 时计算 add(a)/a 的整数商。Cortex-M0 不使用 UDIV，改用重复减法。

```
  AREA Mean,CODE,READONLY
         THUMB
         (1)_______ add
         EXPORT mean_sum
  mean_sum PROC
         (2)_______ {R4,LR}
         MOVS R4,(3)_______
         (4)_______ add
         MOVS R1,(5)_______
  divide  CMP R0,R4
         (6)_______ finish
         SUBS R0,R0,(7)_______
         ADDS R1,R1,(8)_______
         B divide
  finish  MOVS R0,(9)_______
         POP {R4,(10)_______}
         ENDP
         END
```

来源：实验一 · 任务2；实验一 · 已给 add C 函数

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：

P10 / 混合编程 / 10 空 / 课件校订改编

反复调用 C：6174 黑洞步数

C 函数 uint32_t digitSort(uint32_t n) 已给：保留四位十进制（不足补零），降序数减升序数。输入为 0-9999 且至少两位不同。kap_steps 返回首次到达 6174 所需步数；输入已为 6174 时返回 0。

C调用示例（已给）

```
extern uint32_t kap_steps(uint32_t n);
uint32_t steps = kap_steps(3524u); /* steps=3 */
```

```
AREA Kap,CODE,READONLY
        THUMB
        (1)_______ digitSort
        EXPORT kap_steps
kap_steps PROC
        PUSH {R4-R6,(2)_______}
        MOVS R4,(3)_______
        MOVS R5,(4)_______
        LDR R6,(5)_______
loop    CMP R4,R6
        BEQ done
        MOVS R0,(6)_______
        (7)_______ digitSort
        MOVS R4,R0
        ADDS R5,R5,(8)_______
        B (9)_______
done    MOVS R0,R5
        POP {R4-R6,(10)_______}
        ENDP
        END
```

来源：第5讲 · PDF p53-55；用户补充参考图7 · 给定C函数digitSort；用户补充参考图8 · 混合调用框架

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：

P11 / 混合编程 / 10 空 / 课件校订改编

C 调汇编：安全容量下拼接字符串

void append_text(const char *src,char *dst)。R0=src，R1=dst；两区不重叠，dst 容量足够且均有零结尾。将 src 拼到 dst 末尾并保留结尾零。用显式指针递增满足 Cortex-M0 指令形式。

C调用示例（已给）

```
extern void append_text(const char *src,char *dst);
char dst[32] = "Hello";
append_text("World!",dst); /* dst="HelloWorld!" */
```

```
AREA Append,CODE,READONLY
        THUMB
        (1)_______ append_text
append_text PROC
seek    (2)_______ R2,[R1]
        CMP R2,(3)_______
        BEQ copy
        ADDS R1,R1,(4)_______
        B seek
copy    LDRB R2,(5)_______
        (6)_______ R2,[R1]
        ADDS R0,R0,#1
        ADDS R1,R1,#1
        CMP R2,(7)_______
        (8)_______ copy
        (9)_______ (10)_______
        ENDP
        END
```

来源：第5讲 · PDF p56-58；用户补充参考图9 · C调用者与缓冲容量；用户补充参考图10 · 汇编字符串拼接框架

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：

P12 / 软硬件综合 / 10 空 / 教材与真题变式

GPIO 单次流水灯扫描

接口由题面定义：GPIO_BASE=0x40010000，+0x00 为输出数据，+0x04 为方向（bit=1 输出）；8 个 LED 高有效。C 函数 delay_c(uint32_t ticks) 已给。led_scan 完成 01→02→…→80 后熄灭返回；不要求精准延时。

```
  AREA Led,CODE,READONLY
         THUMB
         (1)_______ delay_c
         EXPORT led_scan
led_scan PROC
         PUSH {R4-R6,(2)_______}
         LDR R4,(3)_______
         MOVS R0,(4)_______
         STR R0,[R4,#4]
         MOVS R5,#1
         MOVS R6,(5)_______
loop     STR (6)_______,[R4]
         MOVS R0,#20
         (7)_______ delay_c
         LSLS R5,R5,(8)_______
         SUBS R6,R6,#1
         (9)_______ loop
         MOVS R0,#0
         STR R0,[R4]
         POP {R4-R6,(10)_______}
         ENDP
         END
```

来源：教材 §5.4.2 · GPIO 流水灯；往届 A 卷 · 编程3 GPIO背景

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：

P13 / 软硬件综合 / 10 空 / 真题校订改编

PWM 周期与 C 调用保护

void pwm_once(uint32_t led,uint32_t duty)。输入 R0=LED位图，R1=0-100 占空比；高有效 GPIO 数据口为 0x40010000。delay_pwm_c(ticks) 已给，允许 ticks=0。先亮 duty，再灭 100-duty，完成一个概念 PWM 周期。

```
AREA Pwm,CODE,READONLY
        THUMB
        (1)_______ delay_pwm_c
        EXPORT pwm_once
pwm_once PROC
        PUSH {R4-R6,(2)_______}
        MOVS R4,(3)_______
        MOVS R5,(4)_______
        LDR R6,(5)_______
        STR R4,(6)_______
        MOVS R0,R5
        (7)_______ delay_pwm_c
        MOVS R0,#0
        STR R0,[R6]
        MOVS R0,(8)_______
        SUBS R0,R0,(9)_______
        BL delay_pwm_c
        POP {R4-R6,(10)_______}
        ENDP
        END
```

来源：往届 A 卷 · 编程3 PWM；教材 §4.2.4 · 混合编程

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：

P14 / 软硬件综合 / 10 空 / 实验二改编

实验二：开关值分解为三位十进制

void switch_digits(uint8_t *out)。R0 指向至少3字节RAM，按百、十、个位写出数值0-9。SWITCH_DATA=0x40000000，仅低8位有效。不使用除法指令，采用重复减法。

```
  AREA Digits,CODE,READONLY
         THUMB
         EXPORT switch_digits
  switch_digits PROC
         LDR R1,(1)_______
         LDR R1,[R1]
         MOVS R2,(2)_______
         ANDS R1,R1,R2
         MOVS R2,#0
  hund   CMP R1,(3)_______
         BCC hund_done
         SUBS R1,R1,#100
         ADDS R2,R2,#1
         B hund
  hund_done STRB R2,(4)_______
         MOVS R2,#0
  tens   CMP R1,(5)_______
         (6)_______ tens_done
         SUBS R1,R1,(7)_______
         ADDS R2,R2,#1
         B tens
  tens_done STRB R2,(8)_______
         STRB R1,(9)_______
         BX (10)_______
         ENDP
         END
```

来源：实验二要求；实验二 · 十进制转换与显示流程

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：

P15 / 软硬件综合 / 10 空 / 实验二改编

实验二：按段选与位选完成一帧扫描

void scan_frame(const uint8_t *codes)。R0 指向百、十、个位的段码（已译码，表按输入定义提供）；SEG_LED=0x40000010，SEG_SEL=0x40000014，段高有效、位低有效。位选依次0x0B、0x0D、0x0E；delay_c(20)已给。本题统一显示三位，不消前导零。

```
  AREA Scan,CODE,READONLY
         THUMB
         IMPORT delay_c
         EXPORT scan_frame
  scan_frame PROC
         PUSH {R4-R6,LR}
         SUB SP,SP,#8
         MOVS R4,R0
         LDR R5,(1)_______
         ADR R6,selects
         MOVS R0,#3
         STR R0,[SP]
  next   MOVS R0,(2)_______
         STR R0,[R5,(3)_______]
         (4)_______ R0,[R4]
         STR R0,(5)_______
         LDRB R0,[R6]
         STR R0,[R5,#4]
         MOVS R0,#20
         (6)_______ delay_c
         ADDS R4,R4,#1
         ADDS R6,R6,#1
         LDR R0,[SP]
         SUBS R0,R0,(7)_______
         STR R0,[SP]
         (8)_______ next
         MOVS R0,#15
         STR R0,[R5,#4]
         ADD SP,SP,(9)_______
         POP {R4-R6,(10)_______}
         ALIGN
  selects DCB 0x0B,0x0D,0x0E
         ENDP
         END
```

来源：实验二 · 地址和位选定义；实验二 · 先关后开扫描

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：

P16 / 软硬件综合 / 10 空 / 实验三改编

实验三：带超时与清理的加速器驱动

int trig_hw_run(int32_t angle,int32_t *s,int32_t *c)。R0/R1/R2传参，输出指针合法且字对齐。基址0x40000000；ANGLE +0、CTRL +4、STATUS +8、SIN +0x0C、COS +0x10；CTRL bit0 START、bit1 ABORT；STATUS bit0 BUSY、bit1 DONE、bit2 ERROR。成功返回0，超时-1，硬件错误-2；仅成功写输出，失败先ABORT。

```
  AREA Trig,CODE,READONLY
          THUMB
          EXPORT trig_hw_run
  trig_hw_run PROC
          PUSH {R4-R6,LR}
          LDR R3,(1)_______
          MOVS R6,(2)_______
          STR R6,[R3,#4]
          STR R0,[R3,(3)_______]
          MOVS R6,#1
          STR R6,[R3,#4]
          LDR R5,=4096
  poll    LDR R4,[R3,(4)_______]
          MOVS R6,(5)_______
          TST R4,R6
          BNE error
          MOVS R6,(6)_______
          TST R4,R6
          BNE success
          SUBS R5,R5,#1
          (7)_______ poll
          MOVS R0,#1
          RSBS R0,R0,#0
          B cleanup
  error   MOVS R0,#1
          MVNS R0,R0
  cleanup MOVS R6,#2
          STR R6,[R3,#4]
          B ret
  success LDR R6,[R3,(8)_______]
          STR R6,[R1]
          LDR R6,[R3,(9)_______]
          STR R6,[R2]
          MOVS R0,#0
  ret     POP {R4-R6,(10)_______}
          ENDP
          END
```

来源：实验三 · 任务2与任务3；实验三 · AHB加速器寄存器；实验三 · 汇编驱动接口

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：

P17 / 软硬件综合 / 10 空 / 课堂要求变式

读 Verilog 译码后编写握手驱动

读给定硬件片段，补全一次查询式交易。端口不等待，总线地址相位已锁存。reg_id=0为写入数据，1为控制（写1启动），2为状态（bit0完成），3为结果。接口调用：R0输入，R1输出指针；最多100次状态读取，返回0成功、-1超时。代码须依据译码算出窗口基址与寄存器字节偏移。

```
; 已给硬件（只阅读，不填 Verilog）：
; assign HSEL = (HADDR[31:8] == 24'h400003);
; 地址相位：reg_id <= HADDR[3:2];
; 数据相位case(reg_id)：0 DATA，1 CTRL，2 STATUS，3 RESULT
          AREA Map,CODE,READONLY
          THUMB
          EXPORT mapped_run
  mapped_run PROC
          LDR R2,(1)_______
          STR R0,[R2,(2)_______]
          MOVS R0,(3)_______
          STR R0,[R2,(4)_______]
          MOVS R3,#100
  wait    LDR R0,[R2,(5)_______]
          LSRS R0,R0,#1
          (6)_______ ready
          SUBS R3,R3,#1
          BNE wait
          MOVS R0,#1
          RSBS R0,R0,#0
          BX LR
  ready   LDR R0,[R2,(7)_______]
          STR R0,(8)_______
          MOVS R0,(9)_______
          BX (10)_______
          ENDP
          END
```

来源：重点复习记录 · 39:45-44:22 读硬件写汇编；实验三 · 地址相位/数据相位实例；教材 §5.2 · 全译码与映射

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：

P18 / 软硬件综合 / 10 空 / 课堂方案变式

中断方案：读取结果、清源与设置标志

课堂拓展的中断方案，和实验三轮询方案分开。假定IRQ=1已正确使能并向量绑定：结果端口0x4000040C；清中断端口0x40000410写1后外设撤销IRQ。result与done是C端已定义的volatile uint32_t全局量。处理函数不调用其他函数，只用硬件自动堆栈的R0-R3；读取结果后清中断源，再置done=1并异常返回。

```
  AREA Irq,CODE,READONLY
         THUMB
         (1)______ result
         IMPORT done
         (2)______ ACC_IRQHandler
  ACC_IRQHandler PROC
         LDR R0,(3)______
         (4)______ R1,[R0]
         LDR R2,(5)______
         STR R1,(6)______
         LDR R0,=0x40000410
         MOVS R1,(7)______
         STR R1,[R0]
         LDR R2,(8)______
         STR R1,[R2]
         DSB
         (9)______ (10)______
         ENDP
         END
```

来源：重点复习记录 · 26:16-30:36 中断与加速器；教材 §6.1.2 与 §6.3.2 · 中断

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：

P19 / 混合编程 / 5 空 / 补充参考变式

C 调汇编：任意长度数组的升序检查

C 原型 uint32_t is_sorted(const int32_t *a,uint32_t n)。检查非严格升序，返回1或0；比较按有符号32位数处理。R0=a，R1=n。n为0或1时返回1；n=0不解引用a，n>0时a指向至少n个有效且字对齐的元素。n=5对应参考题，函数也必须处理其他长度。

C调用示例（已给）

```
extern uint32_t is_sorted(const int32_t *a,uint32_t n);
int32_t a[] = {-3,-3,0,8,8};
uint32_t ok = is_sorted(a,5u); /* ok=1 */
```

```
AREA Sorted,CODE,READONLY
        THUMB
        EXPORT is_sorted
is_sorted PROC
        CMP R1,#1
        BLS success
        SUBS R1,R1,#1
        LDR R2,[R0]
loop    ADDS R0,R0,#4
        LDR R3,[R0]
        CMP R2,R3
        (1)_______ fail
        MOVS R2,(2)_______
        SUBS (3)_______,R1,#1
        (4)_______ loop
success MOVS R0,(5)_______
        BX LR
fail    MOVS R0,#0
        BX LR
        ENDP
        END
```

来源：用户补充参考图1 · 长度5的升序题意；用户补充参考图2 · 相邻元素比较框架；教材 §4.2.4 · C/汇编调用

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____

寄存器规划 / 调试记录：

P20 / 混合编程 / 10 空 / 补充参考变式

C 调汇编：两个数组逐项取小取大

C 原型 void split_minmax(uint32_t *a,uint32_t *b,uint32_t n)。将每对元素中较小值写回a[i]、较大值写回b[i]，按无符号32位数比较。R0=a、R1=b、R2=n；0≤n≤65535，两数组有效、字对齐且不重叠，n=0不访存。数组可写；参数指针的数值与数组元素内容不可混淆。

C调用示例（已给）

```
extern void split_minmax(uint32_t *a,uint32_t *b,uint32_t n);
uint32_t a[] = {12,35,0}, b[] = {24,25,0};
split_minmax(a,b,3u); /* a={12,25,0}, b={24,35,0} */
```

```
AREA Pairwise,CODE,READONLY
        THUMB
        (1)_______ split_minmax
split_minmax PROC
        PUSH {R4-R6,(2)_______}
        LSLS R6,R2,(3)_______
        MOVS R3,#0
        CMP R2,#0
        BEQ done
loop    LDR R4,(4)_______
        LDR R5,[R1,R3]
        (5)_______ R4,R5
        (6)_______ keep
        STR R5,(7)_______
        STR R4,[R1,R3]
keep    ADDS R3,R3,(8)_______
        CMP R3,R6
        (9)_______ loop
done    POP {R4-R6,(10)_______}
        ENDP
        END
```

来源：用户补充参考图3 · 两数组对应位置比较；用户补充参考图4 · 字数组与地址偏移框架；教材 §4.2.4 · 调用与寄存器保存

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：

P21 / 混合编程 / 5 空 / 补充参考变式

C 调汇编：用移动掩码统计32位1

C 原型 uint32_t count_mask(uint32_t value)。R0传入任意32位值，以1、2、4、…、0x80000000逐位测试，R0返回1的个数。输入数据放R2保持不变。仅填汇编；不能把参考题的“R7输入、R5输出”直接当成C调用规则。

C调用示例（已给）

```
extern uint32_t count_mask(uint32_t value);
uint32_t bits = count_mask(0x80007001u); /* bits=5 */

AREA MaskCount,CODE,READONLY
        THUMB
        EXPORT count_mask
count_mask PROC
        MOVS R2,R0
        MOVS R0,#0
        MOVS R1,#1
loop    MOVS R3,R2
        ANDS R3,R3,R1
        (1)_______ next
        ADDS R0,R0,(2)_______
next    LSLS R1,R1,(3)_______
        (4)_______ loop
        BX (5)_______
        ENDP
        END
```

来源：用户补充参考图5 · R7/R5位计数题意；用户补充参考图6 · 移动掩码思路；往届 A 卷 · 位统计背景

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____

寄存器规划 / 调试记录：

P22 / 混合编程 / 10 空 / 补充参考变式

迁移题：C 调汇编统计字符出现次数

C 原型 uint32_t count_char(const char *s,uint32_t ch)。s为有效、零结尾的字符串；ch在0-255。仅统计终止零之前与ch相等的字节，不改写字符串；ch=0时返回0。R0=s，R1=ch；R0返回次数。例：count_char("banana",'a')返回3。

C调用示例（已给）

```
extern uint32_t count_char(const char *s,uint32_t ch);
uint32_t n = count_char("banana",(uint32_t)'a'); /* n=3 */

AREA CharCount,CODE,READONLY
        THUMB
        (1)_______ count_char
count_char PROC
        MOVS R2,(2)_______
loop    (3)_______ R3,[R0]
        CMP R3,(4)_______
        BEQ (5)_______
        CMP R3,R1
        (6)_______ skip
        ADDS R2,R2,(7)_______
skip    ADDS R0,R0,(8)_______
        B loop
done    MOVS R0,(9)_______
        BX (10)_______
        ENDP
        END
```

来源：用户补充参考图9 · C调用汇编的字符串背景；用户补充参考图10 · 字节访存与结尾零；C调用汇编迁移题

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：

P23 / 混合编程 / 10 空 / 补充参考变式

迁移题：C 调汇编完成带容量的字符串复制

C 原型 int32_t copy_z(const char *src,char *dst,uint32_t cap)。R0=src，R1=dst，R2=cap，0≤cap≤65535。src有效且零结尾，cap>0时dst至少有cap字节且不与src重叠。完整复制时返回字符数（不含结尾零）；空间不足返回-1，cap>0时仍保证dst零结尾。cap=0不写dst并返回-1。只填汇编，不填写C。

C调用示例（已给）

```
extern int32_t copy_z(const char *src,char *dst,uint32_t cap);
char dst[4];
int32_t n = copy_z("abc",dst,sizeof dst); /* n=3 */
```

```
AREA CopyZ,CODE,READONLY
        THUMB
        (1)_______ copy_z
copy_z PROC
        PUSH {R4,LR}
        MOVS R4,(2)_______
        CMP R2,(3)_______
        BEQ fail
loop    (4)_______ R3,[R0]
        CMP R3,#0
        (5)_______ terminate
        CMP R2,(6)_______
        BEQ truncated
        (7)_______ R3,[R1]
        ADDS R0,R0,#1
        ADDS R1,R1,#1
        ADDS R4,R4,(8)_______
        SUBS R2,R2,#1
        B loop
terminate STRB R3,[R1]
        MOVS R0,(9)_______
        B done
truncated MOVS R3,#0
        STRB R3,[R1]
fail    MOVS R0,#1
        RSBS R0,R0,#0
done    POP {R4,(10)_______}
        ENDP
        END
```

来源：用户补充参考图9 · 原题目标数组容量不足；用户补充参考图10 · 从字节复制迁移到带容量接口

填写答案

(1) _____ (2) _____ (3) _____

(4) _____ (5) _____ (6) _____

(7) _____ (8) _____ (9) _____

(10) _____

寄存器规划 / 调试记录：